

Phase 2: The Continuous Mesh & The Ratchet Effect

The Scenario

Now that your agents can talk and align, you face the next hurdle of ASI: transient memory. Today's multi-agent setups are forgetful; when an API session drops or a pipeline closes, their immediate contextual learning vanishes.

To cross into superintelligence, the system requires a Cognition Fabric - a secure, distributed mesh that stores collective memory. We need to unlock the ratchet effect: a mechanism where if Agent A discovers a useful optimization or mitigation strategy, that insight can be propagated and retained so the network does not have to solve the same edge-case problem again.

Your Task

Extend your Phase 1 prototype into a focused Cognition Fabric demonstration. You are not expected to build an internet-scale system. A compact but credible working prototype, simulation, or mocked architecture demonstration is acceptable if it clearly proves the concept.

Your Phase 2 solution should demonstrate the following core capabilities:

1. A dynamic flow of incoming enterprise-style tasks or incidents that forces agents to make and reuse decisions under changing conditions.
2. A Fabric layer, such as a decentralized ledger, distributed state cache, or equivalent shared memory mechanism, that captures a localized breakthrough from one agent.
3. An Accelerator mechanism that can propagate a verified insight from one agent or node into shared collective memory.
4. An Autonomous Guardrail that checks whether a proposed update violates policy, security, or compliance rules before it is accepted into the Fabric.
5. Reuse of a previously captured insight by another agent, node, or later task so that the ratchet effect is visibly demonstrated.

For this phase, depth matters more than breadth. It is better to show one convincing end-to-end propagation scenario than to sketch many features without proving them.

Bonus Challenge - Chaos Injector: Include a lightweight script or mechanism that simulates a network fault, bad update, or policy violation and demonstrates how the system detects the issue, blocks or repairs the bad propagation, and preserves the integrity of the Fabric.

Strategic Goals to Deliver (The Final Defence)

Your implementation will be judged live through code, demo, and architectural defence. Your code and demo should back up your answers to these hard design dilemmas:

Validating Innovation vs. Hallucination: When an agent claims it has found a novel or optimal solution, how does your Cognition Engine distinguish between a genuine breakthrough and a convincing but invalid hallucination before broadcasting it to the global Fabric? How did you design your verification logic or consensus?

Mitigating Cognitive Drift & Poisoning: A rogue or compromised agent could attempt to inject malicious or low-quality optimizations into the Fabric. How does your architecture detect, isolate, and prune poisoned cognitive updates without resetting the collective memory back to zero?

Managing Fabric Consistency at Scale: If two agents discover conflicting fixes that are valid in different contexts, how does your Fabric preserve contextual consistency without collapsing into split-brain behavior? Did you prioritize eventual consistency, scoped consistency, semantic locking, or another model? Explain the trade-offs of your choice.

You do not need to solve every distributed-systems challenge exhaustively. What matters is a compact, defensible prototype with a clear end-to-end story, a believable memory-sharing mechanism, and visible guardrail logic.